

Full Stack Developer Task: Review Website Module

Objective

Build a simple review platform where users can view products/services and submit reviews.

The goal is to evaluate your skills in:

- Next.js frontend development
 - FastAPI backend development
 - REST API design
 - PostgreSQL database integration
 - Clean code practices
-

Project Requirements

Backend (FastAPI + PostgreSQL)

Create a REST API using FastAPI.

Database Models

User

Fields:

- id
- name
- email
- created_at

Product

Fields:

- id
- title
- description
- image_url
- created_at

Review

Fields:

- id
 - product_id
 - user_id
 - rating (1-5)
 - comment
 - created_at
-

API Requirements

Product APIs

Get all products

```
GET /api/products
```

Response:

```
[
  {
    "id": 1,
    "title": "Laptop",
    "description": "Gaming laptop",
    "average_rating": 4.5
  }
]
```

Get product details with reviews

```
GET /api/products/{id}
```

Response:

```
{
  "id": 1,
  "title": "Laptop",
  "reviews": [
    {
```

```
    "user": "John",
    "rating": 5,
    "comment": "Excellent product"
  }
]
```

Review APIs

Create Review

POST /api/reviews

Request:

```
{
  "product_id": 1,
  "user_id": 2,
  "rating": 5,
  "comment": "Very good"
}
```

Update Review

PUT /api/reviews/{id}

Delete Review

DELETE /api/reviews/{id}

Frontend (Next.js)

Create a responsive review website.

Pages Required

Home Page

Display:

- Product cards
- Product image
- Product name
- Average rating
- Number of reviews

Example:

| Laptop |

| ★★★★★☆ 4.5 |

| 120 Reviews |

| View Details |

Product Details Page

Show:

- Product information
- Average rating
- All user reviews
- Review submission form

Review form:

Fields:

- Name
- Rating (1-5)
- Comment

After submitting:

- Refresh review list
 - Update average rating
-

Technical Requirements

Frontend

Must use:

- Next.js
- JavaScript/TypeScript
- React components
- API integration using fetch/axios
- Proper loading and error handling
- Responsive UI

Backend

Must use:

- FastAPI
 - Python
 - PostgreSQL
 - SQLAlchemy ORM
 - REST API structure
 - Proper validation
-

Bonus Features

(Optional)

Authentication

Implement:

- Register
- Login
- JWT authentication

Admin Panel

Admin can:

- Add products
- Remove products
- Delete inappropriate reviews

Search & Filter

Allow users to:

- Search products
 - Filter by rating
-

Submission Requirements

The candidate should provide:

1. GitHub repository URL
 2. Live frontend URL (Next.js)
 3. Live backend API URL (FastAPI)
 4. API documentation link (Swagger/OpenAPI)
 5. Setup instructions
 6. Database migration/schema
 7. Environment configuration example (.env.example)
-

Expected completion time:

48 hours